PROG IV: Javascript

Profª. Tainá Isabela

Javascript

O JavaScript é uma das principais linguagens usadas no desenvolvimento front-end, sendo essencial para a criação de interfaces web interativas e dinâmicas. Ele é usado para manipular o HTML e o CSS, permitindo que o usuário possa interagir com os elementos da página de diferentes formas.

Por meio do JavaScript, é possível criar animações, efeitos visuais, transições, validar formulários, entre outras possibilidades. Além disso, o JavaScript pode ser integrado com outras tecnologias front-end, como o React, permitindo a criação de aplicações web ainda mais avançadas.

Javascript

JavaScript é uma linguagem de programação completa — contém todas as funcionalidades clássicas de programação que pode ter visto noutras linguagens de programação (ou pelo menos ter ouvido falar), tais como Variáveis, Loops, e Funções.

JavaScript, quando usado em páginas web funciona geralmente por:

- Obter referências a um ou mais valores, como números ou a elementos na página.
- Fazer algo com esses valores, como somar os números.
- Devolvendo um resultado que pode ser usado para fazer outra coisa mais tarde. Por exemplo, pode querer mostrar a soma desses números na página.

Javascript

JavaScript é uma linguagem de programação completa — contém todas as funcionalidades clássicas de programação que pode ter visto noutras linguagens de programação (ou pelo menos ter ouvido falar), tais como Variáveis, Loops, e Funções.

JavaScript, quando usado em páginas web funciona geralmente por:

- Obter referências a um ou mais valores, como números ou a elementos na página.
- Fazer algo com esses valores, como somar os números.
- Devolvendo um resultado que pode ser usado para fazer outra coisa mais tarde. Por exemplo, pode querer mostrar a soma desses números na página.

Javascript

JavaScript é uma linguagem de programação completa — contém todas as funcionalidades clássicas de programação que pode ter visto noutras linguagens de programação (ou pelo menos ter ouvido falar), tais como Variáveis, Loops, e Funções.

JavaScript, quando usado em páginas web funciona geralmente por:

- Obter referências a um ou mais valores, como números ou a elementos na página.
- Fazer algo com esses valores, como somar os números.
- Devolvendo um resultado que pode ser usado para fazer outra coisa mais tarde. Por exemplo, pode querer mostrar a soma desses números na página.

Javascript

Assim como CSS, podemos escrever códigos em JavaScript diretamente do documento HTML, basta inserir dentro da tag <script> .

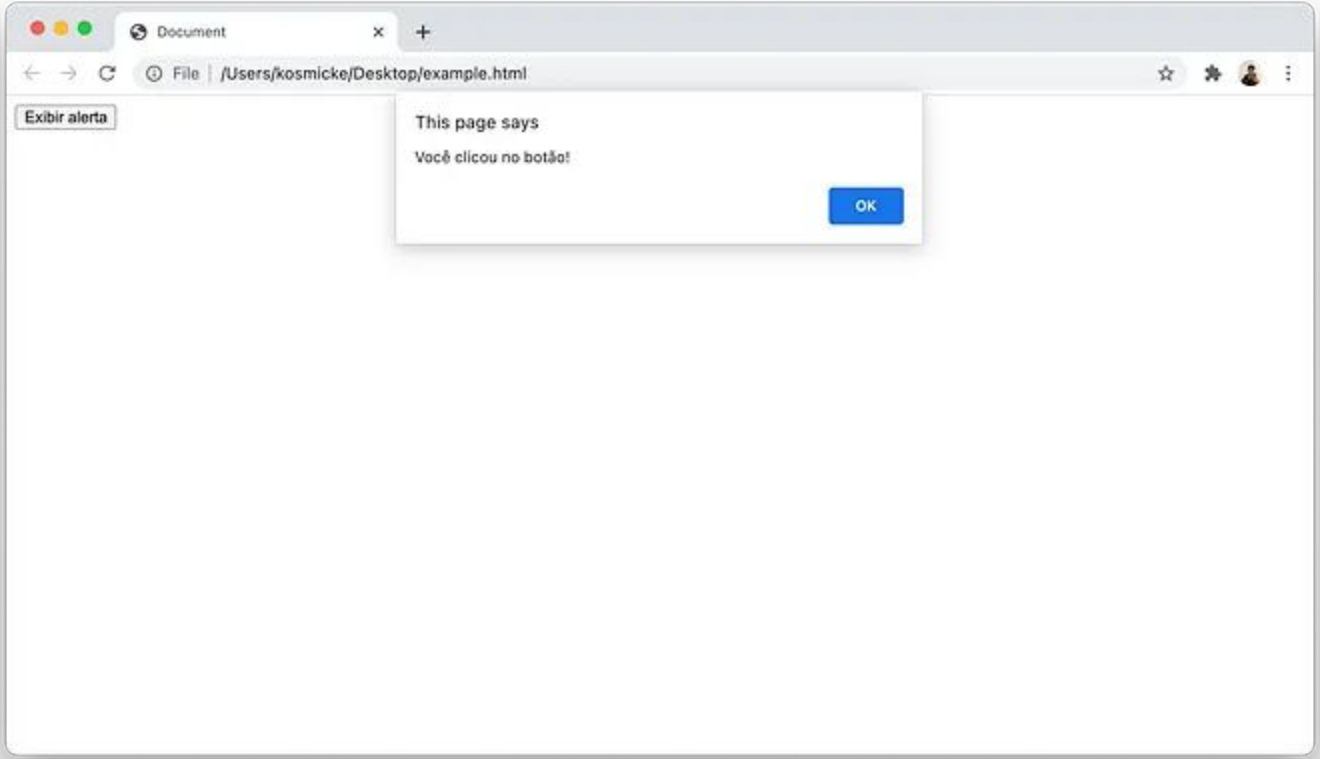
Podemos criar um documento HTML que possui um botão. Este botão, ao ser clicado, executa uma função JavaScript declarada no do bloco de script dentro da página:

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
  </head>
  <body>

  <button onclick="showAlert()">Exibir alerta</button>

  <script>
    function showAlert(){
      window.alert("Você clicou no botão!")
    }
  </script>

  </body>
</html>
```


Document
File
/Users/kosmicke/Desktop/example.html
Exibir alerta
This page says
Você clicou no botão!
OK

JavaScript — Arquivo externo

Outra forma de se utilizar JavaScript em nossa página é colocarmos o script em um arquivo externo, com extensão ".js" e importá-lo em nosso HTML:

```html
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <meta name="viewport" content="width=device-width, in
        <title>Document</title>
    </head>
    <body>

    <button onclick="showAlert()">Exibir alerta</button>

    <script src="script.js"></script>

    </body>
</html>
```

Importação do script

```javascript
function showAlert(){
    window.alert("Você clicou no botão!")
}
```

Sintaxe da linguagem JavaScript

JavaScript foi criada com base na ECMAScript e sua sintaxe é bastante semelhante a linguagens de alto nível muito utilizadas como C e Java. Portanto a sintaxe básica é muito similar como laços condicionais (IF Else), Laços de repetição (For, While, Do While) e Tipos de variáveis (Int, Float, Double, String, Boolean). Uma de suas destoantes diferenças são o tipo de escopo definido para as variavéis:

- let → pode mudar
- const → não muda
- var → global (evitar)

```
let nome = "Ana";
const idade = 20;
var cidade = "São Paulo";
```

Sintaxe da linguagem JavaScript

JavaScript fornece também suporte a funções, aliado às facilidades da tipagem dinâmica, o que torna a definição de métodos simples e prática. Para criar funções, utilizamos a palavra reservada function.

As funções podem receber parâmetros e retornar valores, mas o tipo de retorno e o tipo dos parâmetros não precisa ser previamente definido.

```
function exibirMensagem()
{
  alert("Olá, seja bem vindo(a)!");
}
```

Exibindo mensagens para o usuário com JavaScript

A linguagem JavaScript possui um número imenso de funções nativas que precisariam de vários artigos para serem explicadas detalhadamente. Aqui, serão mostradas apenas algumas das mais utilizadas por quem está iniciando os estudos.

A primeira função apresentada é a alert, que serve para exibir uma mensagem em uma janela popup. Essa função recebe apenas um parâmetro, que é o texto a ser exibido.

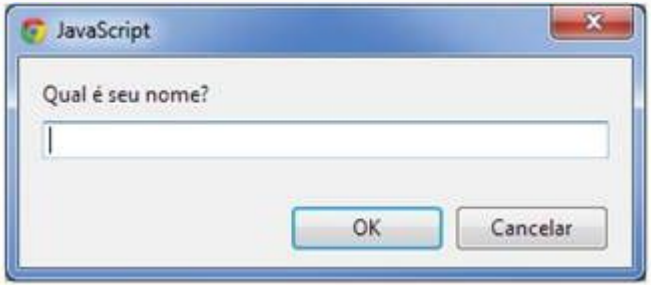
Exibindo mensagens para o usuário com JavaScript

```html
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <script type="text/javascript">
      alert("Olá, seja bem vindo ao Linha de Código.")
    </script>
  </head>
  <body>
  </body>
</html>
```

Exibindo mensagens para o usuário com JavaScript

Em seguida, temos a função prompt, que também abre uma janela popup, mas com uma caixa de texto para coletar informações do usuário. O retorno dessa função é o texto digitado.

```html
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <script type="text/javascript">
      var nome;
      nome = prompt("Qual é seu nome?");
      alert("Olá, " + nome);
    </script>
  </head>
  <body>
  </body>
</html>
```



Manipulando eventos

A linguagem JavaScript também permite trabalhar com eventos dos elementos HTML como botões e caixas de texto. Eventos são disparados quando alguma ação é executada, como o clique e num botão ou a digitação de valores em um input.

No código JavaScript pode-se atribuir valores aos eventos como se fossem propriedades, desde que o valor atribuído seja um código a ser executado.

```html
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
  </head>
  <body>
<button onclick="alert('Você clicou no botão');">clique aqui</button>
  </body>
</html>
```

Manipulando eventos - Externo

```
const listItems = document.querySelectorAll("li");

function toggleDone(e) {
  if (!e.target.className) {
    e.target.className = "done";
  } else {
    e.target.className = "";
  }
}

listItems.forEach((item) => {
  item.addEventListener("click", toggleDone);
});
```

Manipulando eventos

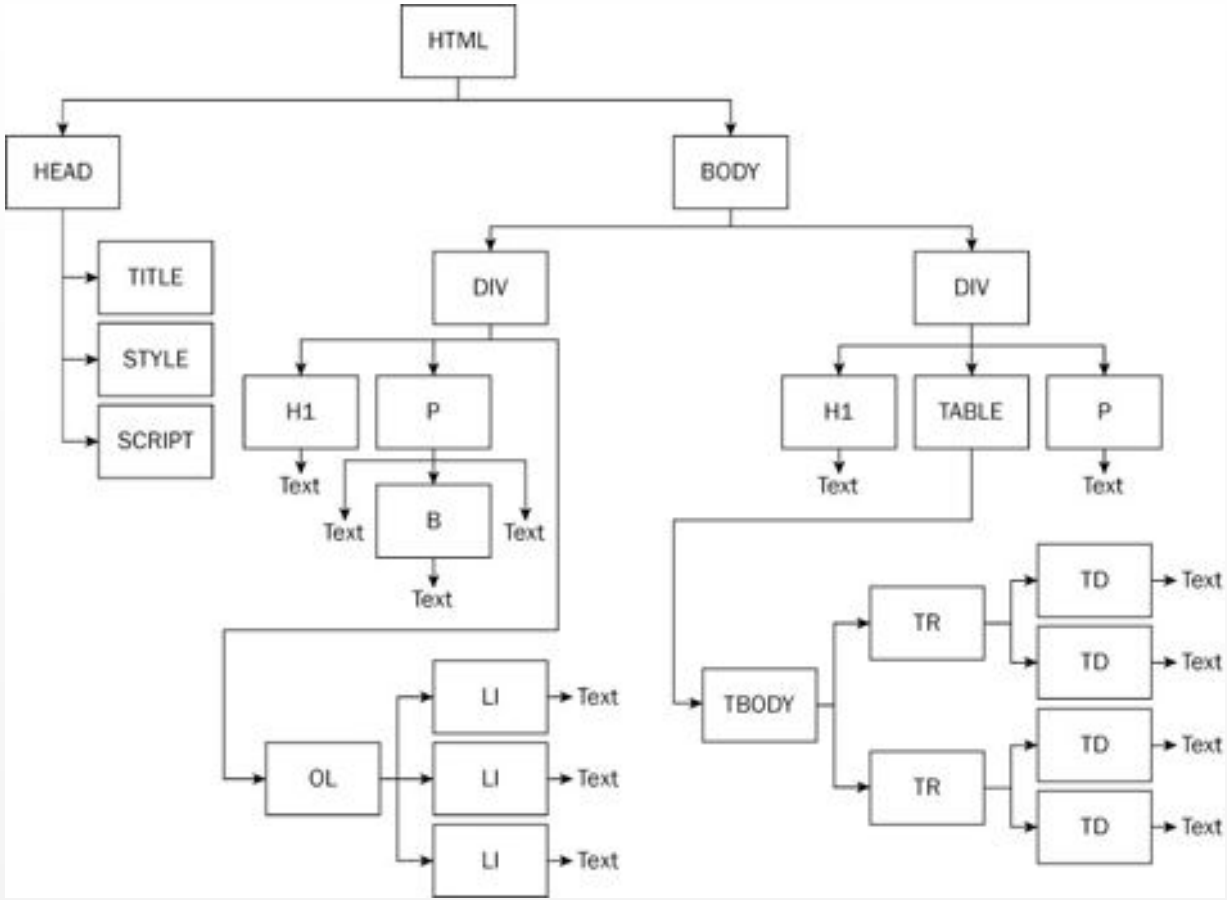
A linguagem JavaScript também permite trabalhar com eventos dos elementos HTML como botões e caixas de texto. Eventos são disparados quando alguma ação é executada, como o clique e num botão ou a digitação de valores em um input.

No código JavaScript pode-se atribuir valores aos eventos como se fossem propriedades, desde que o valor atribuído seja um código a ser executado.

DOM (Document Object Model)

O Modelo de Objeto de Documento (DOM) liga páginas web a scripts ou linguagens de programação ao representar a estrutura de um documento — como o HTML que representa uma página web — na memória. Normalmente refere-se ao JavaScript, embora modelar documentos HTML, SVG ou XML como objetos não faça parte da linguagem JavaScript central.

O DOM representa um documento com uma árvore lógica. Cada ramo da árvore termina num nó, e cada nó contém objetos. Os métodos DOM permitem acesso programático à árvore. Com eles, pode alterar a estrutura, o estilo ou o conteúdo do documento.


HTML
HEAD
BODY
TITLE
STYLE
SCRIPT
DIV
DIV
H1
P
H1
TABLE
P
Text
Text
B
Text
Text
Text
Text
Text
OL
LI
LI
LI
Text
Text
Text
TBODY
TR
TR
TD
TD
TD
TD
Text
Text
Text
Text

DOM (Document Object Model)

O Modelo de Objetos do Documento (DOM) é uma interface de programação para documentos web. Representa a página para que os programas possam alterar a estrutura, o estilo e o conteúdo do documento. O DOM representa o documento como nós e objectos; Assim, as linguagens de programação podem interagir com a página.

Uma página web é um documento que pode ser apresentado na janela do navegador ou como fonte HTML. Em ambos os casos, é o mesmo documento, mas a representação do Modelo de Objeto do Documento (DOM) permite que seja manipulado. Como representação orientada a objetos da página web, pode ser modificada com uma linguagem de scripting como JavaScript.

DOM (Document Object Model)

O DOM não é uma linguagem de programação, mas sem ela, a linguagem JavaScript não teria qualquer modelo ou noção de páginas web, documentos HTML, documentos SVG e as suas partes componentes.

O documento como um todo, o cabeçalho, as tabelas dentro do documento, os cabeçalhos das tabelas, o texto nas células das tabelas e todos os outros elementos de um documento são partes do modelo de objeto do documento para esse documento.

Todos podem ser acedidos e manipulados usando o DOM e uma linguagem de script como JavaScript.

Interfaces DOM

- AbortController
- AbortSignal
- AbstractRange
- Attr
- CDATASection
- CharacterData
- Comment
- CustomEvent
- Document
- DocumentFragment
- DocumentType
- DOMError Deprecated
- DOMException
- DOMImplementation
- DOMParser
- DOMTokenList
- Element
- Event
- EventTarget
- HTMLCollection
- MutationObserver
- MutationRecord
- NamedNodeMap
- Node
- NodeIterator
- NodeList
- ProcessingInstruction
- QuotaExceededError
- Range
- ShadowRoot
- StaticRange
- Text
- TreeWalker
- XMLDocument
- XPathEvaluator
- XPathExpression
- XPathResult
- XSLTProcessor

Interfaces e objetos

Muitos objetos implementam várias interfaces diferentes. O objeto tabela, por exemplo, implementa um objeto especializado HTMLTableElement que inclui métodos como createCaption e insertRow.

Mas como também é um elemento HTML, table implementa o Element interface descrita no DOM Element Capítulo de referência. E finalmente, como um elemento HTML é também, no que diz respeito ao DOM, um nó na árvore de nós que compõem o modelo de objeto para uma página HTML ou XML, o objeto tabela também implementa o mais básico Node da qual Element derivados.

Seleção de elementos (DOM)

document.querySelector()

```
document.querySelector(".classe");
```

Seleciona o primeiro elemento que encontrar.

document.querySelectorAll()

```
document.querySelectorAll("p");
```

Seleciona todos os elementos (retorna lista).

Seleção de elementos (DOM)

document.getElementById()

```
document.getElementById("titulo");
```

Seleciona pelo ID (rápido e direto).

Eventos (interatividade)

addEventListener()

```
botao.addEventListener("click", () => {
  alert("Clicou!");
});
```

Escuta ações do usuário (clique, teclado, mouse).

removeEventListener()
Remove um evento (menos usado, mas importante).

Manipulação de conteúdo

textContent

```
elemento.textContent = "Novo texto";
```

Altera apenas o texto.

innerHTML

```
elemento.innerHTML = "<strong>Texto</strong>";
```

Permite HTML dentro (cuidado com segurança).

Manipulação de estilo

style

```
elemento.style.color = "red";
```

Altera CSS direto no elemento.

classList

```
elemento.classList.add("ativo");
elemento.classList.remove("ativo");
elemento.classList.toggle("ativo");
```

Melhor prática para mexer em estilos.

Criar e remover elementos

createElement()

```
let div = document.createElement("div");
```

Cria novos elementos.

appendChild()

```
document.body.appendChild(div);
```

Adiciona no HTML.

Criar e remover elementos

remove()

```
elemento.remove();
```

Remove elemento.

Iteração

forEach()

```
lista.forEach(item => {
  console.log(item);
});
```

Percorre listas (NodeList, arrays).

Iteração

map()

```
let novo = array.map(x => x * 2);
```

Transforma dados (muito usado com APIs).

Manipulação de dados

JSON.parse()

```
JSON.parse('{"nome":"Ana"}');
```

Converte texto → objeto.

JSON.stringify()

```
JSON.stringify(objeto);
```

Converte objeto → texto.

Requisições (API)

fetch()

```
fetch("https://api.com")
  .then(res => res.json())
  .then(data => console.log(data));
```

Busca dados de servidores.

Temporizadores

setTimeout()

```
setTimeout(() => {
  console.log("Executou");
}, 2000);
```

Executa uma vez depois de um tempo. (2000 ms = 2 segundos)

- Delay de animação
- Mensagens temporárias
- Simular carregamento

Temporizadores

setInterval()

```
setInterval(() => {
  console.log("Repetindo");
}, 1000);
```

Executa várias vezes em loop com intervalo fixo.

- Relógios
- Atualizações automáticas
- Jogos
- Monitoramento

```
clearInterval(intervalo);
```

Exemplo:

```
let botao = document.querySelector("button");

botao.addEventListener("click", () => {
  document.body.style.background = "black";
});
```

Fluxo mental do front-end:

Usuário → Evento → JavaScript → DOM → Atualiza tela

`Clique → addEventListener → função → muda texto → usuário vê resultado`

- Como eu pego um elemento?
  - **querySelector**
- Como eu reajo ao usuário?
  - **addEventListener**
- Como eu mudo a tela?
  - **textContent**, **classList**
- Como eu crio coisas novas?
  - **createElement**

Fontes:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
- https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- https://www.devmedia.com.br/guia/javascript/34372
- https://www.w3schools.com/js/js_intro.asp
- https://dom.spec.whatwg.org/

Questionário